

27 Matrix Find a specific pair in matrix

```
#include <bits/stdc++.h>
using namespace std;
class Solution {
public:
    int findMaxValue(vector<vector<int>>& mat, int N) {
        if (N <= 0) return 0;
        // maxArr[i][j] will store maximum element in submatrix (i..N-1, j..N-1)
        vector<vector<int>> maxArr(N, vector<int>(N, INT_MIN));
        // start from bottom-right
        maxArr[N-1][N-1] = mat[N-1][N-1];
        // last row
        for (int j = N-2; j >= 0; --j) {
            maxArr[N-1][j] = max(mat[N-1][j], maxArr[N-1][j+1]) }
        // last column
        for (int i = N-2; i >= 0; --i) {
            maxArr[i][N-1] = max(mat[i][N-1], maxArr[i+1][N-1]);
        }
        int maxDiff = INT_MIN;
        // fill rest and compute max difference using maxArr[i+1][j+1]
        for (int i = N-2; i >= 0; --i) {
            for (int j = N-2; j >= 0; --j) {
                // candidate difference using bottom-right region
                int candidate = maxArr[i+1][j+1] - mat[i][j];
                if (candidate > maxDiff) maxDiff = candidate;
                // compute maxArr[i][j]
                int curMax = mat[i][j];
                curMax = max(curMax, maxArr[i][j+1]); // right
                curMax = max(curMax, maxArr[i+1][j]); // down
                curMax = max(curMax, maxArr[i+1][j+1]); // diag
                maxArr[i][j] = curMax;
            }
            if no valid pair found (N==1) maxDiff remains INT_MIN; problem expects something sensible
            if (maxDiff == INT_MIN) return 0;
        }
        return maxDiff;
    }
};
```

Compilation Results

Custom Input

Y.O.G.I. (AI Bot)

Problem Solved Successfully

Suggest Feedback

Test Cases Passed

180 / 180

Attempts : Correct / Total

1 / 1

Accuracy : 100%

Points Scored

8 / 8

Time Taken

0.5

Your Total Score: 102

Solve Next

Longest Increasing Path in a Matrix

Minimum swap required to convert binary tree to binary search tree

Maximum number of overlapping intervals

```
11 // start from bottom-right
12 maxArr[N-1][N-1] = mat[N-1][N-1];
13 // Last row
14 for (int j = N-2; j >= 0; --j) {
15     maxArr[N-1][j] = max(mat[N-1][j], maxArr[N-1][j+1]);
16 }
17 // Last column
18 for (int i = N-2; i >= 0; --i) {
19     maxArr[i][N-1] = max(mat[i][N-1], maxArr[i+1][N-1]);
20 }
21
22 int maxDiff = INT_MIN;
23 // fill rest and compute max difference using maxArr[i+1][j+1]
24 for (int i = N-2; i >= 0; --i) {
25     for (int j = N-2; j >= 0; --j) {
26         // candidate difference using bottom-right region
27         int candidate = maxArr[i+1][j+1] - mat[i][j];
28         if (candidate > maxDiff) maxDiff = candidate;
29
30         // compute maxArr[i][j]
31         int curMax = mat[i][j];
32         curMax = max(curMax, maxArr[i][j+1]); // right
33         curMax = max(curMax, maxArr[i+1][j]); // down
34         curMax = max(curMax, maxArr[i+1][j+1]); // diag
35         maxArr[i][j] = curMax;
36     }
37 }
38
39 // if no valid pair found (N==1) maxDiff remains INT_MIN; problem expects som
40 if (maxDiff == INT_MIN) return 0;
41 return maxDiff;
42 }
43
44 };
```